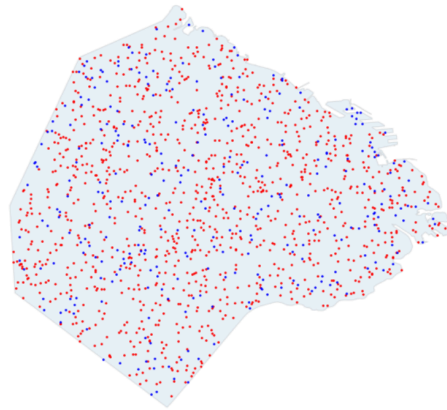


G.A.P Problem

Problema de Asignación Generalizada



Integrantes:

- Iñaki Irarrazaval
- Lisandro Russo
- Manuel Ferrario

Profesores:

- Ivo Pajor
- Juan Pablo Lebon
- Juan Ignacio Castore

1.0 Introducción al problema

Actualmente, ThunderPack (empresa de logística) opera bajo una modalidad descentralizada donde cada vendedor elige libremente, y sin previo aviso, el depósito al que llevará sus productos. Los vendedores suelen encontrarse con depósitos saturados, lo que los obliga a divagar por otros destinos buscando depósitos disponibles, aumentando drásticamente la distancia recorrida y empeorando la experiencia del usuario.

Para mitigar ineficiencias, ThunderPack ha decidido migrar hacia una modalidad de logística centralizada. El presente trabajo aborda esta decisión modelando el escenario como un Problema de Asignación Generalizada (GAP). Nuestro objetivo principal como desarrolladores y analistas de algoritmos es desarrollar programas eficientes que permitan automatizar la toma de decisiones logísticas bajo tres premisas fundamentales:

- Asignar a cada vendedor un único depósito de la red.
- Respetar estrictamente la capacidad máxima de almacenamiento de cada punto de recepción.
- Minimizar la distancia total recorrida por los usuarios para garantizar una operación económica y ágil.
- Intentar llegar a soluciones parecidas a las óptimas.

2.0 Descripción del modelo GAP:

El *Problema de Asignación Generalizada* (GAP) consiste en asignar un conjunto de vendedores a un conjunto de depósitos, minimizando el costo total de asignación sujeto a restricciones de capacidad.

Cada depósito tiene una capacidad máxima medida en unidades, cada vendedor genera una demanda al ser asignado a un depósito, y cada asignación posible tiene un costo asociado.

GAP no es un problema P, por lo que no se conoce un algoritmo de tiempo polinomial que lo resuelva de forma exacta en el caso general, lo que motiva el uso de heurísticas,

búsquedas locales y metaheurísticas para obtener soluciones de buena calidad en tiempo razonable.

3.0 Heurísticas constructivas:

Los algoritmos heurísticos o golosos (greedy) mediante distintas estrategias y variantes logran encontrar soluciones posibles, a problemas costosos. La idea de estos algoritmos no es encontrar soluciones óptimas, ya que, para distintas ocasiones es casi imposible encontrarlas. Solamente, se conforman con encontrar una solución buena en la mayoría de los casos. Brevemente, comentaremos qué funciones heurísticas planteamos para el problema GAP de Thunder Pack.

3.1 Heurística 1

Idea

Priorizar la demanda/costo entre el vendedor y el depósito.

Recorre los vendedores uno por uno y busca un depósito válido para cada vendedor.

Para cada par vendedor-depósito, calcula un indicador de “beneficio” (demanda/costo) y asigna el primer depósito que cumple la condición de beneficio mínimo y capacidad suficiente.

Al momento que se llena un depósito bloquearlo (ningún otro vendedor está habilitado a usarlo), y si queda algún vendedor sin depósito, intentar de asignarle uno a pesar de que pueda tener un costo elevado (evitar penalizaciones altas).

Funcionamiento

- El criterio *profit* usa relación demanda/costo para priorizar asignaciones con demanda grande y costo pequeño.
- Esto evita asignar vendedores a depósitos caros o poco rentables si no hay un mínimo beneficio.
- La segunda pasada permite reducir la cantidad de vendedores no asignados cuando quedan depósitos con capacidad disponible.

3.2 Heurística 2

Idea

Implementando un mecanismo de prioridad de cada vendedor. Ordenamos a cada vendedor dependiendo de cuanto nos “sale” no asignarlo. Para vendedores que tienen mucha demanda (muchos paquetes) y altos costos (tiene muchos depósitos lejanos) son vendedores “difíciles”. Al revés, con los “fáciles”.

Asignamos los vendedores fáciles primeros ya que son los que tienen menos paquetes y más depósitos cercanos. Dejando para el final las difíciles con muchos paquetes.

Para cada vendedor también se calcula una lista de sus mejores depósitos (depósitos más cercanos posibles). Se intenta asignarle a cada vendedor un depósito relativamente alto en sus prioridades, mejorando así, la experiencia del usuario.

Por último, como en la primera heurística, intentamos incluir los vendedores que quedaron colgados en algún depósito posible.

Funcionamiento (Asistido por la IA)

- En el primer bloque de código calculamos la prioridad de cada vendedor (demanda promedio / costo mínimo). Calculamos la “complejidad” del vendedor.
- Calculamos también la lista de prioridades de depósitos para cada vendedor (menor costo → mejor depósito).
- Asignamos vendedores más “fáciles” primero y después si algún vendedor queda colgado intentamos asignar un lugar válido (sin restricciones).

3.3 Resultados de algoritmos Heurísticos

Con los dos algoritmos implementados, solamente queda ver sus resultados. Con ayuda de las instancias pasadas por la cátedra corrimos benchmark y encontramos las soluciones.

Grupo	H1 asignados	H1 costo total	H2 asignados	H2 costo total
GAP_a	100%	26.843	100%	12.653

GAP_b	87.4%	40.349	89.8%	26.546
GAP_e	59.1%	12.573.717	39.0%	17.993.779

Como vemos, en las instancias más fáciles como GAP_a (demandas, costos y capacidades muy flexibles) ambas heurísticas pudieron asignar a todos los vendedores. El algoritmo H2 está dando la mejor solución (pensando en costo) para las instancias más pequeñas. Para los datos un poco más complicados en términos de capacidades de depósitos GAP_b ambas heurísticas promedian 88% aprox de asignación. Finalmente, para los datos más realistas y escalables como GAP_e la heurística número 1 da mejores resultados de asignación que la heurística número 2. Esto se puede explicar porque para H2 se prioriza primero llenar depósitos con vendedores con menor costo y demanda. Así, llenando los “mejores” depósitos disponibles. En cambio, en H1, distribuye mejor y con la segunda vuelta asigna vendedores sin depósito.

Conclusiones con resultados:

H1 → Más distribución, menor vendedores sin depósito. Asigna sin tanta inteligencia. Peor costo.

H2 → Mejor foco en costo para casos donde los depósitos tienen capacidades flexibles. Puede dejar vendedores fuera.

Aclaraciones:

- La primera heurística fue implementada al 100% por el grupo. La segunda guiamos la IA para una resolución un poco más rebuscada y específica.
- El código tiene comentado que pasa en las distintas partes del código.
- En los archivos gap.cpp, gap.h y main.cpp pasamos los txt con las instancias a una clase de instancia y a otra clase de solución.

4.0 Operadores de Búsqueda Local

Después de implementar las funciones heurísticas, planteamos mejorar las soluciones dadas con algoritmos de búsquedas locales. Con los resultados válidos de las heurísticas tenemos oportunidad de mejora. Para esto pensamos estos operadores para analizar cambios de relocate y swap generar una mejor solución si estos pasan un criterio de aceptación.

4.1 Búsqueda Local 1

Idea:

Para nuestra primera implementación de búsqueda local decidimos usar un estilo de relocate que nos pareció intuitivo. Exploramos vendedores y su vecino. Primero, pensamos en implementar switch, pero, cuando lo escribimos quedó como un relocate con parecidos a un switch entre pares vecinos.

Para cada vendedor, vimos que posibilidades tenía. Estar asignado o no estar asignado. Si el vendedor no estaba asignado, intentamos asignar el depósito del vecino. En el caso que esté asignado, vemos si entra el depósito y si cumple que de esa manera baja el costo total del problema. Si cumple las dos condiciones hacemos relocate. Así para ambos vendedores observados.

Implementación:

- Ver dos vendedores de a pares.
- Si alguno no tiene asignado depósito, intentar incluirlo en el depósito del par.
- Si los dos tienen depósito, veo si moviendo alguno al depósito del vecino, mejoró el costo de la solución.

4.2 Búsqueda Local 2

Idea:

Para nuestra segunda implementación de búsqueda local decidimos utilizar una estrategia de swap. Luego de probar distintas alternativas, concluimos que la implementación debía mantener coherencia con la idea detrás de la heurística 2, ya que nuestra intención es basar la búsqueda local en ese caso particular.

La estrategia de swap nos resultó la más adecuada porque permite explorar situaciones en las que la solución inicial asigna a los vendedores de manera poco eficiente. Esto ocurre porque la heurística prioriza el beneficio individual de cada vendedor, pero la mejor ubicación para uno en particular no siempre conduce a la mejor solución global. En algunos casos, asignar a un vendedor al depósito que más le conviene puede obligar a otro a ocupar un depósito significativamente peor, aumentando el costo total de la solución.

En este contexto, el swap permite intercambiar las asignaciones de dos vendedores cuando dicho cambio, aunque pueda empeorar la situación individual de uno de ellos, produce una mejora mayor para el otro y, en consecuencia, reduce el costo total del problema. De esta forma, la búsqueda local puede escapar de óptimos locales generados por decisiones puramente individuales y acercarse a una solución global más eficiente.

Implementación (Asistida por IA):

- Para cada vendedor, intercambiarlo de depósito con los demás vendedores, aumentando el costo puntual, y ver si baja el costo total.
- Si el intercambio lleva a una solución que no es posible, o sea que no entra uno de los vendedores en el respectivo depósito asignado, descartar este intercambio.
- Si el intercambio lleva a una solución factible, y mejora el costo de la solución. Intercambiarlos y actualizar las variables de la solución para los casos siguientes.
- Si ningún intercambio entre vendedores mejora o si sobrepasa el tiempo de timeout, el código corta la búsqueda y entrega la mejor solución que encontró.

3.3 Resultados de Búsquedas Locales

Con la implementación de los dos operadores de búsquedas locales, corremos benchmarks con los ejemplos de gap dados por la cátedra.

Grupo	H1 %	H1 \$	H2 %	H2 \$	H1+ BL1 %	H1+ BL1 \$	H2+ BL2 %	H2+ BL2 \$
GAP_a	100%	26.843	100%	12.653	100%	14.112	100%	12.485
GAP_b	87.4%	40.349	89.8%	26.546	87.4%	39.321	89.8%	25.642
GAP_e	59.1%	12 millon	39%	17 millon	64.3%	11 millon	39.0%	17 millon

Aclaración: \$ → Costo y % → Porcentaje de asignación.

Basamos H1 para BL1 y H2 para BL2. Como se ve en los resultados, no vimos muchas mejoras con implementando los algoritmos de búsqueda local sobre las heurísticas.

En instancias chicas como GAP_a tenemos mejoras de costo para la H1 + BL1, ya que, H1 asigna mucho vendedor pero sin mucha inteligencia. De otra manera, H2 funciona con más inteligencia de asignación, y H2 + BL2 no impacta demasiado en los resultados.

Para las dos instancias siguientes GAP_b y GAP_e vemos mínimas diferencias aplicando las búsquedas locales. Igualmente, para implementar la última función metaheurística necesitaremos la ayuda de las búsquedas locales.

Aclaraciones:

- La primera BL fue implementada al 100% por el grupo. La segunda guiamos la IA para una resolución un poco más rebuscada y específica.
- El código tiene comentado que pasa en las distintas partes del código.
- Corrimos benchmarks solamente con H1+BL1 y H2+BL2.
- BL2 tiene un timeout de sesenta segundos por si no encuentra nunca mejora.

Conclusiones con resultados:

BL1 → Relocate que ayuda para asignar vendedores colgados y optimizar un poco el costo. Contra: solo compara entre pares. En instancias grandes no ayuda.

BL2 → Reduce muchos costos cuando la solución inicial tiene mucho margen (mucha capacidad y muchos vendedores). Contra: no hace nada con los vendedores colgados.

4.0 Algoritmos Metaheurísticos

Los algoritmos metaheurísticos son implementaciones un poco más complejas de lo que fuimos viendo en heurísticas y búsquedas locales. Su objetivo es encontrar soluciones suficientemente buenas para problemas de optimización complejos. Hay dos formas de encontrar las soluciones para estos problemas, la primera es la explotación y la segunda es la exploración. El método de explotación examina muchas veces los alrededores de las mejores soluciones, tratando de encontrar mínimos locales cercanos. La exploración tiene otro enfoque que se basa en ir más lejos de la solución encontrada y tratar de encontrar mínimos lejanos.

4.1 Implementación de Algoritmo Metaheurístico

Idea:

Repasando las maneras de implementar los problemas metaheurísticos nos llamó la atención el algoritmo de GRASP (Greedy Randomized Adaptive Search Procedure). Ya que, para las implementaciones que teníamos anteriormente nos quedaron muchos vendedores sin depósito y costos muy altos. Queriendo mejorar las soluciones para instancias reales como las de GAP_e.

El algoritmo corre en loop por un número arbitrario de sesenta segundos, arma una lista de depósitos óptimos para cada vendedor. Los depósitos se ordenan de mayor a menor profit (demanda/costo, como en las heurísticas). Luego, se arma un RCL

(Restricted Candidate List) con tamaño rcl size que se pasa por parámetro. Este número afecta al “random” del algoritmo. Si el número es muy bajo, tenemos un algoritmo muy greedy (goloso), ya que, quiere ver una solución demasiado buena. Después, de esa lista, agarra un depósito random para construir una solución posible y asignarse al vendedor. Esto sería la parte de exploración de la implementación.

Dentro del loop todavía tenemos una fase de reparación y de búsqueda local. La fase de reparación consiste en tratar de asignar vendedores de manera random para después correr nuestro algoritmo de búsqueda local dos (explotación) e intentar mejorar la solución.

Este ciclo se repite por sesenta segundos.

Implementación:

- Timeout por iteración de sesenta segundos.
- Recorro todos los vendedores, les asignó depósitos óptimos.
- De los depósitos óptimos agarro rcl size mejores depósitos y le asignó uno random a cada vendedor.
- Se corre un ciclo de reparación para asignar vendedores que quedaron colgados.
- Si la solución es mejor a la que teníamos, la actualizamos a la mejor.

4.3 Resultados de Metaheurísticas

Finalmente, para resumir todo el trabajo, volvimos a testear los resultados de nuestros algoritmos sobre la instancia real de GAP. Vimos resultados que nos sorprendieron bastante.

Tabla de resultados en instancia real:

Algoritmo	Asignados	Costo
H1	1100/1100 (100%)	10.498
H1+BL1	1100/1100 (100%)	10.498

Meta (GRASP) (rcl=3)	1100/1100 (100%)	415
H2+BL2	1100/1100 (100%)	250
H2	1100/1100 (100%)	306

Analizando la tabla final, vemos que, los mejores resultados los dieron el **H2+BL2**. Dando incluso mejor que en algoritmos más complejos como el GRASP. Estos resultados se dan porque armamos la heurística dos y la búsqueda local dos para que funcionen bien conjuntamente. Por un lado, H2 asigna bien para depósitos flexibles donde hay mucho espacio y la BL2 reduce mucho el costo para estos casos. Haciendo una buena combinación de ambas.

También, la metaheurística no fue lo que esperábamos. Intentamos también subir un poco el rcl size a cinco dando muy poco margen de mejora comparado al tamaño tres que habíamos pensado previamente. Igualmente, para casos donde se complica la asignación como en GApE vimos que GRASP dio buenos resultados.

Tabla de resultados GRASP en GApE:

Asignación promedio	Costo promedio
58.8%	1.131.713

5.0 Conclusiones

En conclusión, los algoritmos implementados funcionan mejor o peor dependiendo el caso de GAP. Si tuviéramos que elegir dos algoritmos para implementar en casos reales serían **H2+BL2** y también **GRASP**. Siendo lo que mejor costo dio en el problema. Igualmente, creemos que el problema tendría más sentido si no solo se penaliza el costo sino que también la demanda. Es decir, podemos encontrar soluciones muy buenas enfocándonos solamente en los vendedores con mayor distancia a los depósitos. Pero, podemos dejar vendedores afuera que tienen muchos paquetes para entregar siendo un problema para una empresa como ThunderPack.

Aclaración: El .zip contiene un README.md con las instrucciones de ejecución, uso de IA, compilación, etc.